Iris Species Classification Using a Gaussian Naive Bayes Classifier Implemented from Scratch

MD. Shams Shahaiar
Student ID: 2203011
Course: Artificial Intelligence CSE-3207
CSE / RUET

October 25, 2026

Abstract

This report presents the implementation and evaluation of a Gaussian Naive Bayes classifier built from scratch using NumPy and applied to the Iris flower dataset. The dataset of 150 records was split into 120 training and 30 testing samples using a stratified 80/20 split. The model estimates class priors, feature means and feature variances from the training data and predicts the species of a flower by maximising the log posterior probability. On the test set the classifier achieved an accuracy of 96.67%, a macro precision of 0.9697, a macro recall of 0.9667 and a macro F1-score of 0.9666. Only one of the 30 test flowers was misclassified (an *Iris-versicolor* predicted as *Iris-virginica*). The results are presented using a confusion matrix heatmap and a bar chart of actual versus predicted class counts.

1 Background Study

1.1 Classification in Machine Learning

Classification is a supervised learning task in which a model learns a mapping from input features to a discrete class label using labelled examples. Once trained, the model assigns labels to previously unseen data. Typical applications include spam filtering, medical diagnosis, document categorisation and species identification. The quality of a classifier is judged by how well its predictions on unseen test data agree with the true labels.

1.2 Bayes' Theorem

The Naive Bayes family of classifiers is based on Bayes' theorem, which relates the posterior probability of a class C_k given a feature vector $\mathbf{x}$ to the likelihood of the features and the prior probability of the class:

$$P(C_k \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid C_k)\, P(C_k)}{P(\mathbf{x})}. \tag{1}$$

Since $P(\mathbf{x})$ is the same for every class, the predicted class is the one that maximises the numerator:

$$\hat{y} = \arg\max_k \; P(C_k) \prod_{i=1}^{n} P(x_i \mid C_k). \tag{2}$$

The product over features follows from the "naive" assumption that all features are conditionally independent given the class. Although this assumption is rarely perfectly true, Naive Bayes often performs well in practice and is very fast to train because it only requires simple statistics to be computed from the data.

1.3 Gaussian Naive Bayes

When the features are continuous, as with the flower measurements in the Iris dataset, the likelihood of each feature is modelled with a normal (Gaussian) distribution whose mean $\mu_{k,i}$ and variance $\sigma_{k,i}^2$ are estimated separately for each class k and feature i:

$$P(x_i \mid C_k) = \frac{1}{\sqrt{2\pi\sigma_{k,i}^2}} \exp\left(-\frac{(x_i - \mu_{k,i})^2}{2\sigma_{k,i}^2}\right). \tag{3}$$

Multiplying many small probabilities can cause numerical underflow, so the classification is performed in log space:

$$\hat{y} = \arg\max_k \left[\log P(C_k) + \sum_{i=1}^{n} \log P(x_i \mid C_k)\right]. \tag{4}$$

1.4 The Iris Dataset

The Iris dataset was introduced by Fisher in 1936 [1] and is one of the most widely used benchmark datasets in pattern recognition. It is available from the UCI Machine Learning Repository [2]. It contains measurements of three iris species and is popular for teaching because it is small, clean and contains one class (*Iris-setosa*) that is linearly separable from the other two.

2 Dataset and Preparation

2.1 Dataset Description

The dataset was loaded directly from the UCI repository into a pandas DataFrame [3]. It contains 150 records with four numeric features and one categorical target:

- `sepal_length` (cm)
- `sepal_width` (cm)
- `petal_length` (cm)
- `petal_width` (cm)
- `species`: *Iris-setosa*, *Iris-versicolor* or *Iris-virginica*

2.2 Data Quality Checks

Fully empty rows were removed, although none were found, so all 150 records were retained. A check for missing values returned zero for every column, so no imputation was required. The class distribution is perfectly balanced, with 50 records for each species. Because of this balance, accuracy is a reliable metric and the class prior probabilities are equal.

Table 1: Class distribution of the full dataset.

Species	**Records**
Iris-setosa	50
Iris-versicolor	50
Iris-virginica	50
Total	**150**

2.3 Train–Test Split

The data were divided into 80% training and 20% testing subsets using `train_test_split` from scikit-learn [5], with `random_state=42` for reproducibility and `stratify=y` so that each subset preserves the class proportions. This produced 120 training records (40 per species) and 30 testing records (10 per species). The four features were used as input (X) and the species column as the target (y). No feature scaling was applied, since Gaussian Naive Bayes models each feature independently and is not sensitive to differences in feature scale.

3 Methodology and Implementation

The classifier was implemented manually with NumPy [4] rather than by calling a library implementation, so that every step of the algorithm is explicit. The implementation consists of four steps.

3.1 Step 1: Training (Parameter Estimation)

For each species c, the training records belonging to that species were selected and three quantities were computed:

- the prior probability $P(c) = N_c/N$, where N_c is the number of training records in class c and N is the total number of training records;
- the mean of each of the four features (`np.mean` along axis 0);
- the variance of each of the four features (`np.var` along axis 0).

Variances were clipped to a minimum of 10^{-9} to prevent division by zero. Because the training set is balanced, each class has a prior of $40/120 = 0.3333$.

3.2 Step 2: Gaussian Probability Function

A function `gaussian_probability(x, mean, variance)` evaluates the normal density in Equation (3) for every feature at once using vectorised NumPy operations.

3.3 Step 3: Prediction

To classify a single flower, the function `predict_one(x)` computes, for each class, a score equal to the log prior plus the sum of the log Gaussian probabilities of the four features (Equation (4)). The class with the highest score is returned. The core of the implementation is shown below.

```
def predict_one(x):
    scores = {}
    for c in classes:
        prior = model[c]["prior"]
        mean = model[c]["mean"]
        variance = model[c]["variance"]

        score = np.log(prior)
```

```
        probs = gaussian_probability(x, mean, variance)
        score += np.sum(np.log(probs))
        scores[c] = score
    return max(scores, key=scores.get)
```

3.4 Step 4: Evaluation

All 30 test flowers were classified with `predict_one`. The predictions were compared with the true labels using accuracy, macro-averaged precision, macro-averaged recall, macro-averaged F1-score and a confusion matrix, all computed with scikit-learn [5]. For a class k, precision is $TP_k/(TP_k + FP_k)$, recall is $TP_k/(TP_k + FN_k)$, and F1 is their harmonic mean. Macro averaging computes each metric per class and then takes the unweighted mean, giving every species equal importance. The visualisations were produced with Matplotlib [6] and Seaborn [7].

4 Results, Metrics and Graphs

4.1 Sample Predictions

The first ten test predictions all matched the actual species (three *Iris-setosa*, four *Iris-versicolor* and three *Iris-virginica*), which gives an early indication that the model separates the classes well.

4.2 Overall Performance Metrics

Table 2 summarises the performance of the classifier on the 30 test records.

Table 2: Overall evaluation metrics on the test set.

Metric	Value
Accuracy	0.9667
Macro Precision	0.9697
Macro Recall	0.9667
Macro F1-score	0.9666

4.3 Confusion Matrix

Figure 1 shows the confusion matrix as a heatmap. Rows are the actual species and columns are the predicted species.

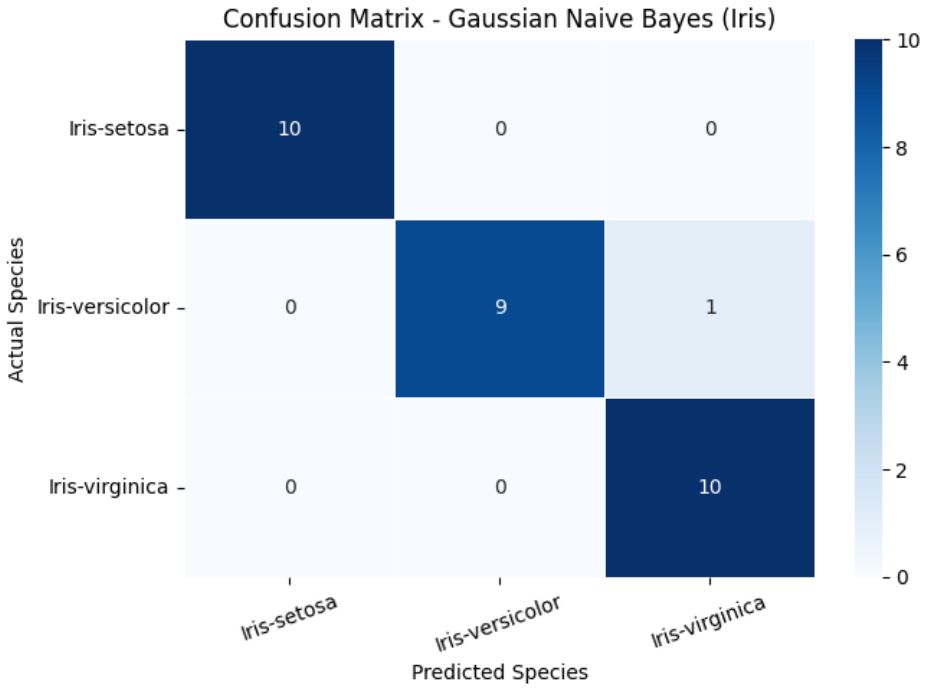


Figure 1: Confusion matrix heatmap of the Gaussian Naive Bayes classifier on the test set.

All 10 *Iris-setosa* and all 10 *Iris-virginica* flowers were classified correctly. Of the 10 *Iris-versicolor* flowers, 9 were correct and 1 was misclassified as *Iris-virginica*. This is the only error, giving 29 correct predictions out of 30.

4.4 Per-Class Performance

Using the confusion matrix, the per-class precision, recall and F1-score are given in Table 3. Their unweighted means equal the macro values in Table 2.

Table 3: Per-class metrics derived from the confusion matrix.

Species	Precision	Recall	F1-score
Iris-setosa	1.0000	1.0000	1.0000
Iris-versicolor	1.0000	0.9000	0.9474
Iris-virginica	0.9091	1.0000	0.9524

Iris-setosa is classified perfectly, which is consistent with its well-known separation from the other two species. *Iris-versicolor* has perfect precision but a recall of 0.90 because one of its flowers was lost to the virginica class. Conversely, *Iris-virginica* has perfect recall but a precision of 0.9091 because it received one extra, incorrect prediction.

4.5 Actual versus Predicted Counts

Figure 2 compares the number of actual and predicted flowers per species.

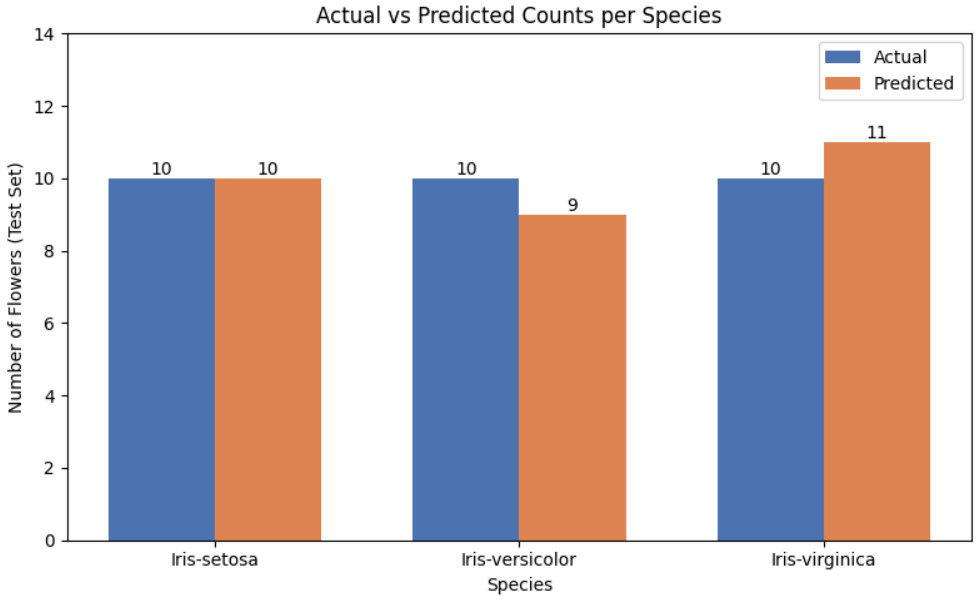


Figure 2: Actual versus predicted counts for each species in the test set.

The actual counts are 10, 10 and 10 for *Iris-setosa*, *Iris-versicolor* and *Iris-virginica*, while the predicted counts are 10, 9 and 11. The difference of one flower in each of the last two classes is caused by the single versicolor flower predicted as virginica.

4.6 Discussion

The results confirm that Gaussian Naive Bayes is an effective classifier for the Iris dataset. The only confusion occurs between *Iris-versicolor* and *Iris-virginica*, whose petal and sepal measurements overlap more than those of setosa. The naive independence assumption is not strictly valid here, since petal length and petal width are strongly correlated, yet the model still performs very well. A limitation of this experiment is the small test set of only 30 samples: a single misclassification changes the accuracy by about 3.3 percentage points, so the result may vary with a different random split. Techniques such as k-fold cross-validation would give a more stable estimate.

5 Conclusion

In this project a Gaussian Naive Bayes classifier was implemented from scratch with NumPy and used to classify the species of Iris flowers. The dataset was complete and balanced, and a

stratified 80/20 split produced 120 training and 30 testing records. The model achieved an accuracy of 96.67%, with macro precision, recall and F1-score of 0.9697, 0.9667 and 0.9666 respectively, misclassifying only one *Iris-versicolor* flower as *Iris-virginica*. The confusion matrix heatmap and the actual-versus-predicted bar chart clearly illustrate this behaviour. The work shows that a simple probabilistic model, built only from class priors, means and variances, can deliver strong results on a well-structured dataset. Future work could include cross-validation, comparison with other classifiers such as k-nearest neighbours or decision trees, and testing on larger and more complex datasets.

References

[1] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936.

[2] D. Dua and C. Graff, "UCI Machine Learning Repository: Iris Data Set," University of California, Irvine, School of Information and Computer Sciences. [Online]. Available: https://archive.ics.uci.edu/ml/datasets/iris

[3] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.

[4] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[7] M. L. Waskom, "seaborn: statistical data visualization," *Journal of Open Source Software*, vol. 6, no. 60, p. 3021, 2021.